CSC3060 **Tutorial 6**

Project 4: Part3
Optimize Replacement Policy and Prefetcher

Xiaoying Zeng

April 7th,9th 2026

1. Commands & Notices About Part 3

- Commands
- Trace Analyzer

2. Prefetcher

- NextLine
- Stride

3. Replacement Policy

- BIP
- SRRIP

1. Commands & Notices About Part 3

- **Generate personalized trace:**

./trace_generator/workload_gen student <student_id> > my_trace.txt

- **Use Trace Analyzer:**

python3 trace_analyzer.py my_trace.txt

- **Get personalized Best_AMAT data and work towards it:**

Open Part3_best_AMAT.xls and find your corresponding AMAT

1. Commands & Notices About Part 3

1.Best_AMAT : The best AMAT result using combination of LRU/BIP/SSRIP and NextLine/Stride Prefetcher

(Of course, you can do better than that!)

2.Baseline_AMAT： AMAT under Part 2 Policy (L1 LRU None + L2 LRU None)

$$\text{Score} = \begin{cases} 50, & \text{Student\_AMAT} < \text{Best\_AMAT} \\ 40, & \text{Student\_AMAT} = \text{Best\_AMAT} \\ 40 \times \dfrac{\text{Baseline\_AMAT} - \text{Student\_AMAT}}{\text{Baseline\_AMAT} - \text{Best\_AMAT}}, & \text{Best\_AMAT} < \text{Student\_AMAT} < \text{Baseline\_AMAT} \\ 0, & \text{Student\_AMAT} \geq \text{Baseline\_AMAT} \end{cases}$$

Suppose Best_AMAT =2, Baseline_AMAT =20:

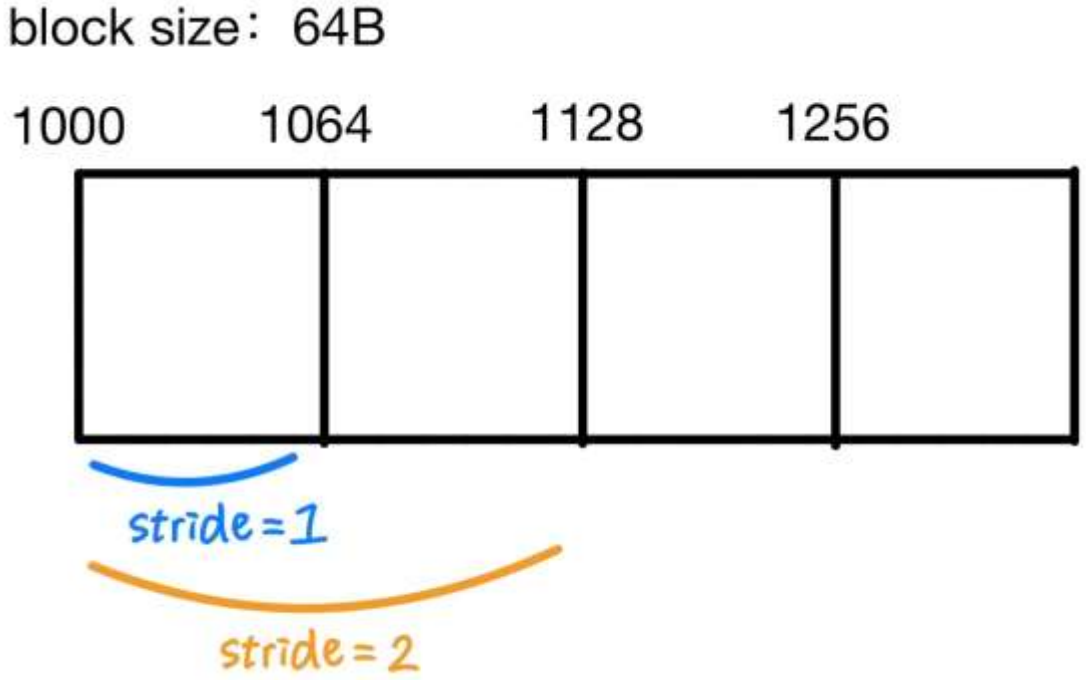
Student AMAT	Interval	Score
1.5	< 2.0	50.0
2.0 (reference Best_AMAT)	= 2.0	40.0
5.0	Middle	33.33
10.0	Middle	22.22
15.0	Middle	11.11
20.0 (reference Baseline_AMAT)	≥ 20.0	0
25.0	≥ 20.0	0

1. Commands & Notices About Part 3

Trace Analyzer:

1. Top block strides：

- If |stride| = 1 is very high, try NextLine prefetcher.
- If a fixed stride is also high, prioritize Stride prefetcher.



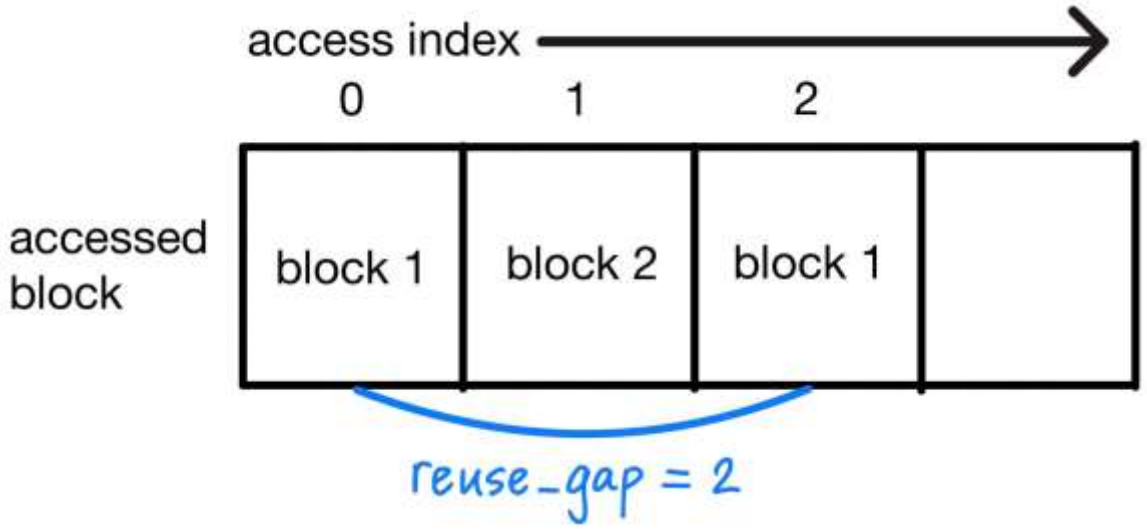

```
Top block strides (all directions)
  1: 3649 (61.20%)
  64: 1434 (24.05%)
  -1088: 54 (0.91%)
  -576: 51 (0.86%)
  -64: 41 (0.69%)
  -448: 38 (0.64%)
  256: 31 (0.52%)
  512: 28 (0.47%)
  -192: 26 (0.44%)
  384: 23 (0.39%)
```

1. Commands & Notices About Part 3

Trace Analyzer:

2.Reuse-gap buckets

- proportion of reuse gaps in 1~16 is high, prioritize recency-based policies (LRU).
- If short reuse is low and scans are frequent, prioritize scan-resistant policies (BIP / SRRIP).




```
Reuse-gap buckets (distance in accesses between repeated blocks)
  257-1024: 2301 (48.38%)
  17-64: 1246 (26.20%)
  5-16: 939 (19.74%)
  65-256: 181 (3.81%)
  2-4: 57 (1.20%)
  1: 29 (0.61%)
  >1024: 3 (0.06%)
Average reuse gap: 311.61 accesses
```

1. Commands & Notices About Part 3

3.Phase Summary: shows the general pattern of trace

Example below:

- Phase 1: Warm-up Period [0-2048)
- Phase 2: Hotspot Period [2048-4096)
- Phase 3: Tail Period [4096-5992]

```
Phase summary (window size = 512 accesses)
  [     0,    512) reads= 470 writes=  42 unique_blocks= 192 repeated= 62.50% dominant_stride=   1 (97.65%) unique_l1_sets=  64
  [   512,   1024) reads= 482 writes=  30 unique_blocks= 512 repeated=  0.00% dominant_stride=   1 (99.80%) unique_l1_sets=  64
  [  1024,   1536) reads= 512 writes=   0 unique_blocks= 512 repeated=  0.00% dominant_stride=   1 (99.80%) unique_l1_sets=  64
  [  1536,   2048) reads= 512 writes=   0 unique_blocks= 512 repeated=  0.00% dominant_stride=   1 (100.00%) unique_l1_sets=  64
  [  2048,   2560) reads= 512 writes=   0 unique_blocks= 247 repeated= 51.76% dominant_stride=   1 (43.44%) unique_l1_sets=  64
  [  2560,   3072) reads= 508 writes=   4 unique_blocks=  34 repeated= 93.36% dominant_stride=  64 (63.41%) unique_l1_sets=   2
  [  3072,   3584) reads= 487 writes=  25 unique_blocks=  20 repeated= 96.09% dominant_stride=  64 (83.37%) unique_l1_sets=   1
  [  3584,   4096) reads= 486 writes=  26 unique_blocks=  20 repeated= 96.09% dominant_stride=  64 (83.37%) unique_l1_sets=   1
  [  4096,   4608) reads= 488 writes=  24 unique_blocks= 366 repeated= 28.52% dominant_stride=   1 (60.86%) unique_l1_sets=  64
  [  4608,   5120) reads= 498 writes=  14 unique_blocks= 451 repeated= 11.91% dominant_stride=   1 (78.08%) unique_l1_sets=  64
  [  5120,   5632) reads= 498 writes=  14 unique_blocks= 451 repeated= 11.91% dominant_stride=   1 (78.08%) unique_l1_sets=  64
  [  5632,   5992) reads= 351 writes=   9 unique_blocks= 320 repeated= 11.11% dominant_stride=   1 (78.27%) unique_l1_sets=  64
```

How do we lower AMAT?

Implement Prefetcher, Replacement Policy

Or change parameters of the cache:
associativity and block size
(Do note that you can't change the cache size or latency)

Prefetcher

2. Prefetcher

NextLine Prefetcher : prefetches the next block

- works well for sequential accesses

- Calculate Prefetch (Pseudo code) :

```
prefetch_addresses = EMPTY_LIST
block_start = (current_address / BLOCK_SIZE) * BLOCK_SIZE
prefetch_addresses.ADD(block_start + BLOCK_SIZE)
RETURN prefetch_addresses
```

2. Prefetcher

Stride Prefetcher : Prefetches blocks at a regular interval (stride)

- works well for fixed-step accesses
- Calculate Prefetch (Common Logic):

```
STATE {
   has_ previous _block
   previous_block
   previous_stride
   confidence(0-3)
}
```

For each incoming address:

1. Convert to block number
2. If there is history record(has_previous_block == TRUE):
   - Calculate stride
   - If stride is stable(stride == previous _stride && stride != 0) → Increase confidence
   - Otherwise → Update stride, reset confidence
   - If confidence ≥ 2 and stride ≠ 0 → Prefetch the next block based on stride
3. Update history record
4. Return prefetch list

2. Prefetcher (possible solution to further lower AMAT:)

StrideFast Prefetcher

Calculate Prefetch Example(Logic):

```
STATE {
   has_ previous _block
   previous_block
   previous_stride
   confidence(0-3)
}
```

For each incoming address:

1. Convert to block number
2. If there is history record(has_previous_block == TRUE):
   - Calculate stride
   - If stride is stable(stride == previous _stride && stride != 0) → Increase confidence
   - Otherwise → Update stride, reset confidence
   - If confidence ≥ 2 and stride ≠ 0 → Prefetch the next block based on stride
   - If (previous_stride > 0) AND (confidence ≥ 1) AND (miss == TRUE)→ Prefetch the next block based on stride
3. Update history record
4. Return prefetch list

Replacement Policy

3. Replacement Policy

BIP(Bimodal Insertion Policy)

Maintains a last_access parameter for each cache line:

- **On miss:** Most new entries are inserted as the oldest (LRU); Occasionally (every 'throttle' misses, usually 32), a new entry is inserted as the newest (MRU)
- **On hit:** move to MRU position
- **On victim selection:** always choose the block with the smallest last_access (oldest/LRU)

Key property: prevents scan pollution by quickly evicting most new blocks, while still giving some blocks a chance to stay

3. Replacement Policy

SRRIP(Static Re-Reference Interval Prediction)

Maintains a Re-Reference Prediction Value (RRPV) for each cache line:

0(very likely) ~ 3(very unlikely)

- OnMiss(Insertion): line.rrpv = 2
- OnHit: line.rrpv = 0
- GetVictim: While (true):

```
For each way in set:
    If (way.rrpv == 3): Return way
For each way in set:
    If (way.rrpv < 3): way.rrpv = way.rrpv + 1
```

3. Replacement Policy (possible solution to lower AMAT:)

```
FUNCTION findLruWay(set)
   victim_index = 0
   oldest_time = set[0].last_access

FOR i FROM 1 TO set.size - 1
      IF set[i].last_access < oldest_time THEN
         oldest_time = set[i].last_access
         victim_index = i
      END IF
   END FOR

   RETURN victim_index
END FUNCTION
```

```
FUNCTION findPrefetchAwareLruWay(set)
   victim_index = -1
   oldest_time = 0

   FOR i FRO
      IF set[i]
         CONT
      END IF

      victir
      oldes
   END IF
   END FOR

   IF victim_index != -1 THEN
      RETURN victim_index
   END IF

   RETURN findLruWay(set)
END FUNCTION
```

ime THEN

Choose Victim:

Pick the oldest blocks

3. Replacement Policy (possible solution to lower AMAT:)

```
FUNCTION findLruWay(set)
   victim_index = 0
   oldest_time = set[0].last_access

FOR i FROM 1 TO set.size - 1
      IF set[i].last_access < oldest_time THEN
         oldest_time = set[i].last_access
         victim_index = i
      END IF
   END FOR

   RETURN victim_index
END FUNCTION
```

```
FUNCTION findPrefetchAwareLruWay(set)
   victim_index = -1
   oldest_time = 0

   FOR i FROM 0 TO set.size – 1
      IF set[i].is_prefetched == false THEN
         CONTINUE
      END IF

      IF victim_index == -1 OR set[i].last_access < oldest_time THEN
         victim_index = i
         oldest_time = set[i].last_access
      END IF
   END FOR

   IF victim_index != -1 THEN
      RETURN victim_index
   END IF

   RETURN findLruWay(set)
END FUNCTION
```

3. Replacement Policy (possible solution to lower AMAT:)

Choose Victim:

Pick the pre-fetched blocks first

avoid prefetched blocks evicting demand blocks

```
FUNCTION findPrefetchAwareLruWay(set)
   victim_index = -1
   oldest_time = 0

   FOR i FROM 0 TO set.size – 1
      IF set[i].is_prefetched == false THEN
         CONTINUE
      END IF

      IF victim_index == -1 OR set[i].last_access < oldest_time THEN
         victim_index = i
         oldest_time = set[i].last_access
      END IF
   END FOR

   IF victim_index != -1 THEN
      RETURN victim_index
   END IF

   RETURN findLruWay(set)
END FUNCTION
```

Thank you

Q&A